```
-- ================================================================
-- IDC backend: tables, row level security, vault functions, storage
-- شغّليه كله مرة واحدة في Supabase > SQL Editor
-- ================================================================
create extension if not exists pgcrypto with schema extensions;

create type item_level as enum ('public','private','secret');
create type user_role  as enum ('student','grad','employee','owner');

-- ---------- 1) البيانات الأساسية: المستخدمين ----------
create table public.profiles (
  id          uuid primary key references auth.users(id) on delete cascade,
  full_name   text not null default '',
  handle      text unique,                 -- بيتحط في رابط الكارت NFC / QR
  governorate text,
  role        user_role not null default 'student',
  photo_path  text,
  onboarded   boolean not null default false,   -- خلّصت شاشة البيانات الأساسية؟
  created_at  timestamptz not null default now()
);

-- بيانات حساسة: صاحبها بس يشوفها (مش بتظهر في الصفحة العامة أبدًا)
create table public.profile_private (
  id          uuid primary key references auth.users(id) on delete cascade,
  national_id text,
  birth_date  date,
  phone       text
);

-- ---------- 2) الجهات المعتمدة ----------
create table public.issuers (
  id       uuid primary key default gen_random_uuid(),
  name     text unique not null,
  name_en  text,
  verified boolean not null default true
);
insert into public.issuers (name, name_en) values
  ('وزارة الداخلية','Ministry of Interior'),
  ('وزارة الاتصالات (ITI)','Ministry of Communications (ITI)'),
  ('جامعة القاهرة','Cairo University'),
  ('Cisco','Cisco'),('Microsoft','Microsoft'),('Coursera','Coursera'),
('Udemy','Udemy');

-- ---------- 3) الوثائق: عامة / خاصة / سرية ----------
create table public.items (
  id           uuid primary key default gen_random_uuid(),
```

```
  user_id      uuid not null default auth.uid() references auth.users(id) on delete
cascade,
  level        item_level not null default 'public',
  type         text not null,
  title        text not null,
  value        text,
  issuer_id    uuid references public.issuers(id),
  issuer_text  text,                        -- جهة مكتوبة بإيدك (غير موثقة)
  secret_until timestamptz,                 -- بعد التاريخ ده السري بيبقى عام
  on_card      boolean not null default false,  -- يظهر في الصفحة العامة للكارت؟
  file_path    text, file_name text, file_kind text,
  created_at   timestamptz not null default now()
);
create index on public.items(user_id, level);

-- ---------- 4) خزنة القسم الخاص ----------
create table public.vault_credentials (
  user_id      uuid primary key references auth.users(id) on delete cascade,
  pw_hash      text not null,
  failed       int  not null default 0,
  locked_until timestamptz,
  updated_at   timestamptz not null default now()
);
create table public.vault_sessions (
  user_id    uuid primary key references auth.users(id) on delete cascade,
  expires_at timestamptz not null
);
create table public.access_logs (
  id      bigserial primary key,
  user_id uuid not null references auth.users(id) on delete cascade,
  action  text not null,
  ok      boolean not null,
  at      timestamptz not null default now()
);

-- ---------- 5) دوال مساعدة ----------
create or replace function public.effective_level(i public.items)
returns item_level language sql stable as $$
  select case when i.level='secret' and i.secret_until is not null and i.secret_until
<= now()
              then 'public'::item_level else i.level end
$$;

-- الخزنة مفتوحة = (اتحقق بخطوتين aal2) + (باسورد الخزنة اتكتب من أقل من دقيقتين)
create or replace function public.vault_open()
returns boolean language sql stable security definer set search_path = public as $$
  select coalesce(auth.jwt()->>'aal','') = 'aal2'
     and exists (select 1 from vault_sessions where user_id = auth.uid() and
expires_at > now())
$$;
```

```
create or replace function public.vault_status()
returns jsonb language sql stable security definer set search_path = public as $$
  select jsonb_build_object(
    'has_vault',    exists (select 1 from vault_credentials where user_id =
auth.uid()),
    'open',         public.vault_open(),
    'locked_until', (select locked_until from vault_credentials where user_id =
auth.uid()))
$$;

-- إنشاء/تغيير باسورد الخزنة. بيرجع: ok | weak | same_as_account | wrong_old | unauthenticated
create or replace function public.set_vault_password(pw text, old_pw text default
null)
returns text language plpgsql security definer set search_path = public, extensions
as $$
declare c vault_credentials; acc text;
begin
  if auth.uid() is null then return 'unauthenticated'; end if;
  if length(pw) < 8 or pw !~ '[0-9]' or pw !~ '[A-Za-z\u0600-\u06FF]' then return
'weak'; end if;
  select encrypted_password into acc from auth.users where id = auth.uid();
  if acc is not null and crypt(pw, acc) = acc then return 'same_as_account'; end if;
  select * into c from vault_credentials where user_id = auth.uid();
  if found and (old_pw is null or crypt(old_pw, c.pw_hash) <> c.pw_hash) then return
'wrong_old'; end if;
  insert into vault_credentials(user_id, pw_hash) values (auth.uid(), crypt(pw,
gen_salt('bf', 10)))
  on conflict (user_id) do update set pw_hash = excluded.pw_hash, failed = 0,
locked_until = null, updated_at = now();
  return 'ok';
end $$;

-- فتح الخزنة: ok | wrong | locked | no_vault. بعد 3 غلط بيتقفل 60 ثانية
create or replace function public.unlock_vault(pw text)
returns text language plpgsql security definer set search_path = public, extensions
as $$
declare c vault_credentials;
begin
  select * into c from vault_credentials where user_id = auth.uid();
  if not found then return 'no_vault'; end if;
  if c.locked_until is not null and c.locked_until > now() then
    insert into access_logs(user_id, action, ok) values (auth.uid(), 'unlock_locked',
false);
    return 'locked';
  end if;
  if crypt(pw, c.pw_hash) = c.pw_hash then
    update vault_credentials set failed = 0, locked_until = null where user_id =
auth.uid();
    insert into vault_sessions(user_id, expires_at) values (auth.uid(), now() +
```

```
interval '2 minutes')
      on conflict (user_id) do update set expires_at = excluded.expires_at;
    insert into access_logs(user_id, action, ok) values (auth.uid(), 'unlock', true);
    return 'ok';
  end if;
  update vault_credentials
     set locked_until = case when failed + 1 >= 3 then now() + interval '60 seconds'
end,
         failed       = case when failed + 1 >= 3 then 0 else failed + 1 end
   where user_id = auth.uid();
  insert into access_logs(user_id, action, ok) values (auth.uid(), 'unlock', false);
  return 'wrong';
end $$;

create or replace function public.lock_vault() returns void
language sql security definer set search_path = public as $$
  delete from vault_sessions where user_id = auth.uid()
$$;
-- بتمدّد الجلسة دقيقتين طول ما انتي شغالة
create or replace function public.touch_vault() returns void
language sql security definer set search_path = public as $$
  update vault_sessions set expires_at = now() + interval '2 minutes'
   where user_id = auth.uid() and expires_at > now()
$$;

revoke all on function public.set_vault_password(text,text),
public.unlock_vault(text),
  public.lock_vault(), public.touch_vault(), public.vault_status() from public, anon;
grant execute on function public.set_vault_password(text,text),
public.unlock_vault(text),
  public.lock_vault(), public.touch_vault(), public.vault_status() to authenticated;

-- صفحة الكارت العامة (NFC/QR): بترجّع بس الاسم والمحافظة والوثائق العامة اللي علّمتيها "على الكارت"
create or replace function public.get_public_profile(h text)
returns jsonb language sql stable security definer set search_path = public as $$
  select jsonb_build_object(
    'full_name', p.full_name, 'governorate', p.governorate, 'role', p.role,
    'items', coalesce((
      select jsonb_agg(jsonb_build_object('type', i.type, 'title', i.title, 'value',
i.value,
               'issuer', coalesce(s.name, i.issuer_text), 'verified',
coalesce(s.verified, false))
             order by i.created_at desc)
        from items i left join issuers s on s.id = i.issuer_id
       where i.user_id = p.id and i.on_card and public.effective_level(i) =
'public'), '[]'::jsonb))
  from profiles p where p.handle = h
$$;
grant execute on function public.get_public_profile(text) to anon, authenticated;
```

```
-- عشوائي handle تلقائيًا (بالاسم اللي كتبه) و profile أول ما حد يسجّل: يتعمل له
create or replace function public.handle_new_user() returns trigger
language plpgsql security definer set search_path = public as $$
begin
  insert into profiles(id, full_name, handle)
  values (new.id, coalesce(new.raw_user_meta_data->>'full_name',''), 'u' ||
substr(md5(random()::text || new.id::text), 1, 8));
  insert into profile_private(id, phone) values (new.id, new.phone);
  return new;
end $$;
create trigger on_auth_user_created after insert on auth.users
  for each row execute function public.handle_new_user();

-- ---------- 6) Row Level Security ----------
alter table public.profiles          enable row level security;
alter table public.profile_private   enable row level security;
alter table public.issuers           enable row level security;
alter table public.items             enable row level security;
alter table public.vault_credentials enable row level security;  -- من غير policies =
ممنوع تمامًا من الواجهة
alter table public.vault_sessions    enable row level security;
alter table public.access_logs       enable row level security;

create policy "own profile read"   on public.profiles for select using (id =
auth.uid());
create policy "own profile update" on public.profiles for update using (id =
auth.uid()) with check (id = auth.uid());
create policy "own private all"    on public.profile_private for all using (id =
auth.uid()) with check (id = auth.uid());
create policy "issuers read"       on public.issuers for select to authenticated
using (true);

-- العام بيظهر دايمًا، الخاص والسري (اللي لسه سري) مش بيظهروا غير والخزنة مفتوحة
create policy "items read"   on public.items for select
  using (user_id = auth.uid() and (public.effective_level(items) = 'public' or
public.vault_open()));
create policy "items insert" on public.items for insert
  with check (user_id = auth.uid() and (level = 'public' or public.vault_open()));
create policy "items update" on public.items for update
  using (user_id = auth.uid() and (public.effective_level(items) = 'public' or
public.vault_open()))
  with check (user_id = auth.uid() and (level = 'public' or public.vault_open()));
create policy "items delete" on public.items for delete
  using (user_id = auth.uid() and (public.effective_level(items) = 'public' or
public.vault_open()));

create policy "own logs read" on public.access_logs for select using (user_id =
auth.uid());

-- ---------- 7) التخزين (صور وملفات PDF) خاص بالكامل ----------
```

```
insert into storage.buckets (id, name, public) values
('documents','documents',false), ('avatars','avatars',false)
on conflict (id) do nothing;

create policy "avatars own" on storage.objects for all
  using (bucket_id = 'avatars' and (storage.foldername(name))[1] = auth.uid()::text)
  with check (bucket_id = 'avatars' and (storage.foldername(name))[1] =
auth.uid()::text);
-- المسار: documents/<user>/<item>/<file>. القراءة مرتبطة بصلاحية الـ item نفسه (يعني الخاص محتاج
الخزنة مفتوحة)
create policy "documents write" on storage.objects for insert
  with check (bucket_id = 'documents' and (storage.foldername(name))[1] =
auth.uid()::text);
create policy "documents read" on storage.objects for select
  using (bucket_id = 'documents' and (storage.foldername(name))[1] = auth.uid()::text
         and exists (select 1 from public.items i where i.id::text =
(storage.foldername(name))[2]));
create policy "documents delete" on storage.objects for delete
  using (bucket_id = 'documents' and (storage.foldername(name))[1] =
auth.uid()::text);
```